BFF: Backend for Frontend

Spring 2025

Introduction

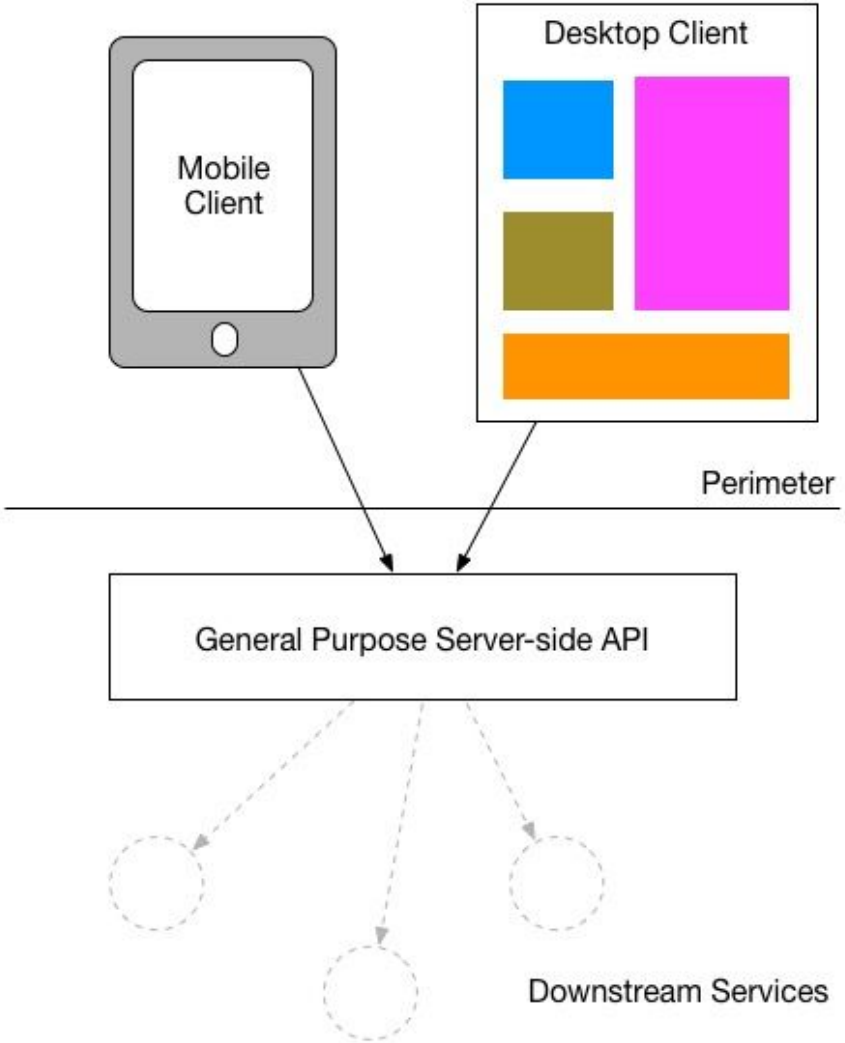
- At first, we created SSR websites.
  - Documents are created on the server and sent to client.
- We were sending the user interface over the web.
- Reduced the cost of releasing new functionality by eliminating client-side installs.

Mobile-era

- Introduction of mobile devices created a new challenge.
- Now, we need to expose server-side functionality via both desktop web UI and mobile UIs.
- Systems developed for desktop-web struggled with new UIs.
- Since there were **tight coupling** between desktop-web UI and backend services, the adaptation were difficult.

General-purpose API

- Creating a general-purpose API is the first step to support both desktop and mobile interfaces.
- Functionality is added over time to meet the needs of the new UIs.
- General-purpose APIs work well if calls are similar across UIs.

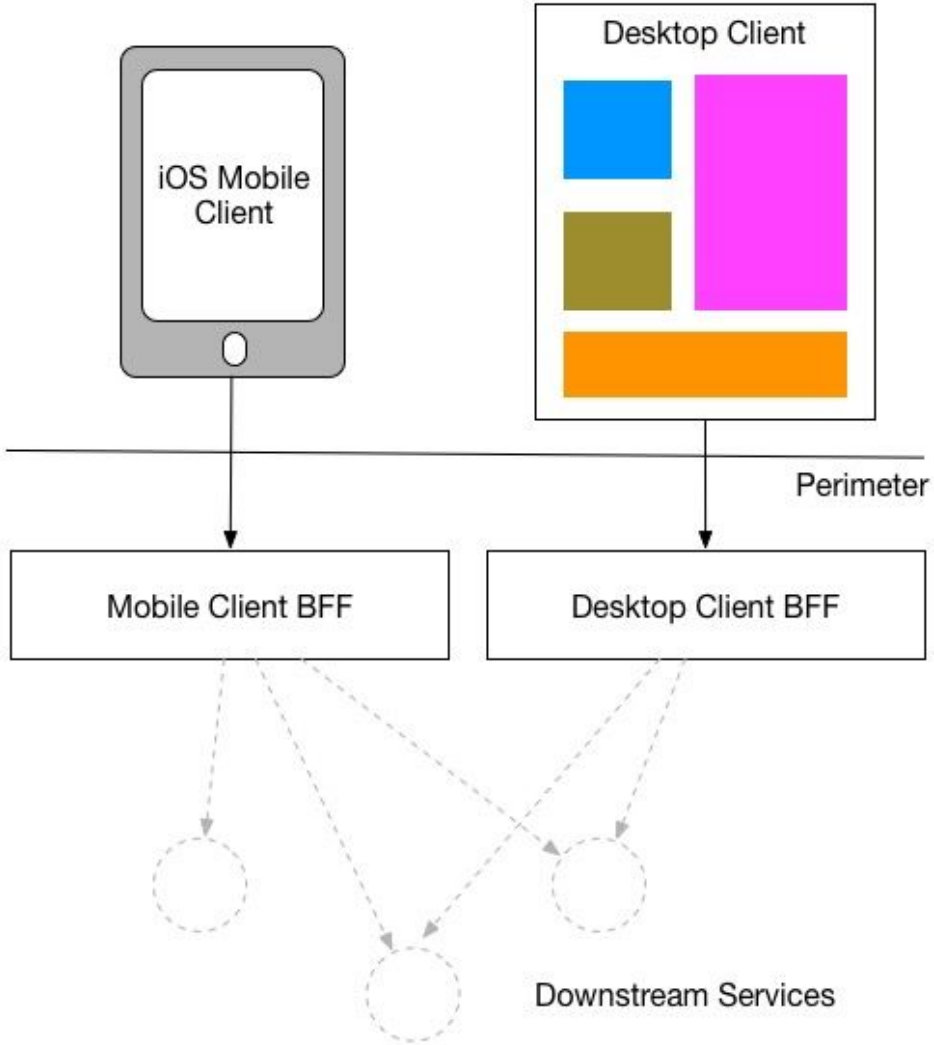


Mobile challenges

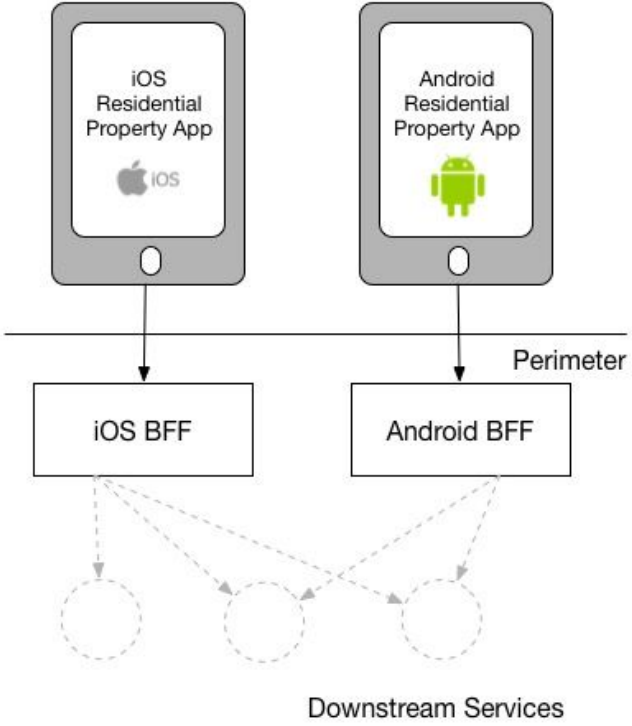
- However, mobile and desktop experiences are different.
- Mobile devices have less screen space.
  - Less data can be displayed.
- Frequent server connections drain battery life.
  - Also, mobile phones have limited data plans.
- Mobile interactions may also differ.
  - e.g. barcode scanners, qr code scanners etc.
- Therefore, in mobile devices we want to make *different*, *fewer* and *simpler* API calls compared to desktop.
  - APIs need additional functionalities to cater to mobile-specific use cases.

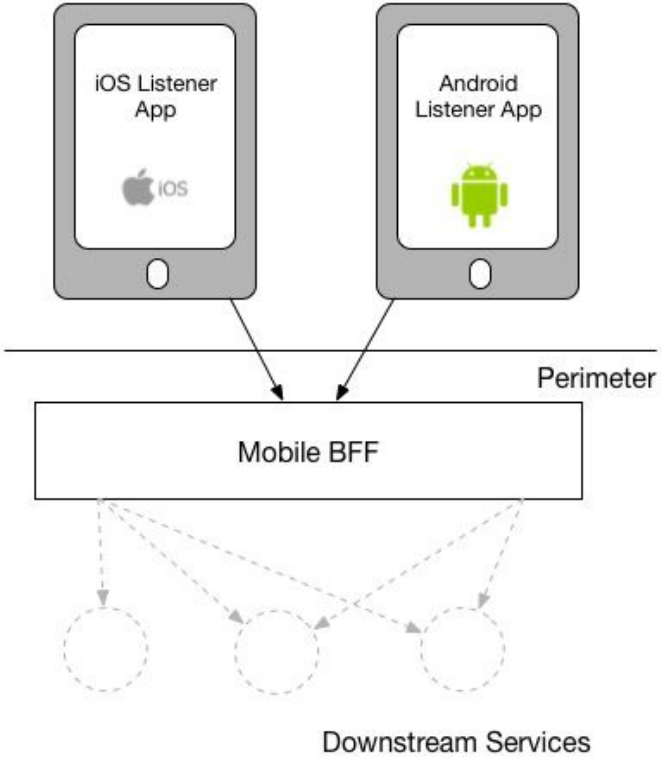
BFF

- An alternative to general-purpose API backend.
- One backend per UX rather than a single, monolithic API.
- Each BFF is designed specifically for a user experience.
  - Smaller, focused backends with minimal overheads.
- BFF is usually maintained by frontend team.
- Speeds up the UI-specific requirements.
  - They can now quickly implement without affecting other interfaces
  - And without changes to the original backend.


Desktop Client
iOS Mobile Client
Perimeter
Mobile Client BFF
Desktop Client BFF
Downstream Services

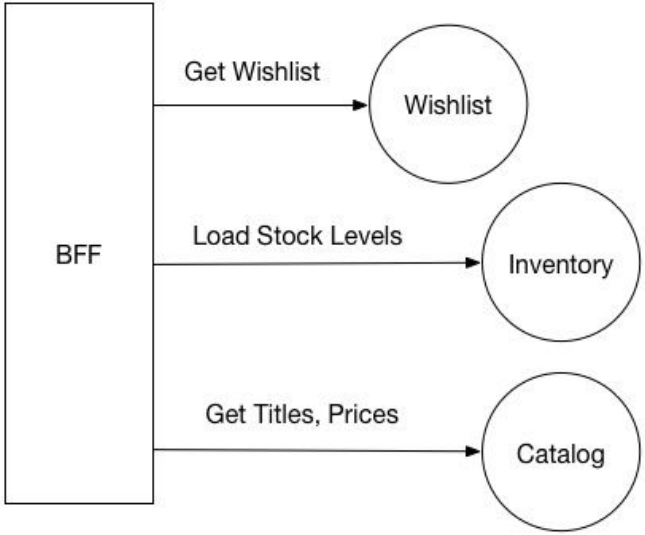
How many BFFs?





- There are multiple ways.
- The stricter way is less risky.
  - Also, **one experience, one bff**
- Since iOS and Android **experiences** are similar, it can be justified using a single BFF.
  - However, if they diverge, use different BFFs.
- It is also said that BFF’s should be aligned around team boundaries.
  - If you have a single mobile team, have a single BFF.
  - If you have separate teams for iOS and Android, have two BFF’s.

The Brakes - Give Blood	In Stock! (14 items remaining)	$5.99	Order Now
Blue Juice - Retrospectable	Out Of Stock	$17.50	Pre Order
Hot Chip - Why Make Sense?	Going fast (2 items left)	$9.99	Order Now



Imagine the scenario above. Here, it is possible that we are taking all these data from different microservices.

- Therefore, a BFF can actually create a method to get them all in a single request coming to itself.
  - The backend is still the same!
- Can be essential for systems with large number of microservices

Summary

- You can think of BFFs as just having a use in solving the constraints of mobile devices.
- The desktop web experience is typically delivered on more powerful devices with better connectivity, where the cost of making multiple downstream calls is manageable.
  - This can allow your web application to make multiple calls directly to downstream services without the need for a BFF.
- However, BFF’s can also be useful for the web.
  - If you are generating the UI on the backend, BFF can be useful for it.
  - Can simplify caching by putting a reverse proxy in front of it.
- BFF’s can be exposed to 3rd parties.
  - 3rd parties usually don’t want to deal with your API. (facebook, google etc)
  - By using BFF, you can just do what they want.

Summary

- Frontend and backend can work in parallel more efficiently.
  - Normally, frontend might be blocked waiting for backend APIs
- With BFF, frontend can build their own dedicated backend which acts as an **intermediary** between frontend and *actual* backend services.
  - Now they can build their own system without waiting for backend.
- So FE builds their UI and uses a mock or placeholder API in BFF.
  - Backend only deals with their API without thinking of the FE.
- Once the backend is ready, BFF can be updated to connect to the actual backend.
  - Much easier compared to the first alternative, because FE and BFF are working together seamlessly at this point!
- Allows teams to decouple frontend and backend development.

- Tailored Data Filtering
  - Receives a full dataset (like 10 fields) but downs it to 5 and sends it.
- Data aggregation
  - Combine data from multiple microservices into a single response
- Mobile Optimization
  - By filtering and aggregating, BFF addresses limited screen space, battery drain from connections, and data plan constraints
- Decoupling
  - Frontend can modify the UI and data requirements without playing with backend.
- Parallel Workflow
  - Frontend can create a BFF with mock data while backend is working.

https://samnewman.io/patterns/architectural/bff/